Imperial College London

Department of Earth Science and Engineering

MSc in Environmental Data Science and Machine Learning

Independent Research Project

Project Plan

Achieving Internal Consistency in Databases used by PHREEQC Aqueous Modeling Software Via A Python Package

By

Joshua Leiper

Email: jjl122@ic.ac.uk

GitHub username: edsml-jjl122

Repository link: https://github.com/ese-msc-2023/irp-jjl122

Supervisors:

Professor Dominik Weiss

Dr. Fengjie Liu

September 2024

Contents

Abstract 3
Introduction 3
Background and Motivation 3
Geochemical Software and their Databases 4
The Coding Challenge 5
Aims and Objectives 5
Methods 6
Data Processing Pipeline 6
A. Data Extraction and Conversion 7
B. Data Cleaning and Standardization 7
C. Database Compilation 8
Data Analysis 8
Solution Master Species (SMS) 9
Solution Species 9
Testing and Validation 9
Results and Discussion 10
Analysis 10
Conflicts in Solution Master Species 10
Attribute Conflicts: Alkalinity and GFW 11
Species Conflicts 11
Conflict in Solution Species 11
Novel Database and Tool Development 12
Limitations 13
Conclusion 14
References 14

Abstract

Geochemical modelling is a tool in environmental science, allowing researchers to simulate interactions in natural systems that are often impractical to study experimentally. PHREEQC, a widely used geochemical modelling software, relies heavily on comprehensive and consistent thermodynamic databases to produce accurate simulations. However, inconsistencies within existing PHREEQC databases, particularly in species definitions and thermodynamic parameters, can lead to significant errors in simulation outcomes. This study presents the development of a Python package designed to enhance the internal consistency of thermodynamic data and aqueous species definitions in PHREEQC databases. By scraping and consolidating data from seven PHREEQC-provided databases, we created a novel, unified master database optimized for speciation calculations.

The Python package automates data extraction, cleaning, and integration processes, addressing common conflicts in species definitions and equilibrium constants. The resulting master database includes a broader range of elements and equations than any individual existing database, facilitating more comprehensive and accurate geochemical simulations. Preliminary validation demonstrates that the new database produces results consistent with those generated using existing databases, with minor differences highlighting areas for further investigation. Despite these advancements, the study acknowledges limitations, including the exclusion of the PHASES block and limited validation simulations. Future work will focus on expanding the database and conducting more extensive validation to ensure broader applicability and robustness.

Introduction

Background and Motivation

Simulation software has become an indispensable tool in the modern scientists repertoire, working in concert with laboratory experiments. Simulations are often a first port-of-call for testing theories and predicting outcomes, allowing researchers to explore a wide range of conditions quickly and cost-effectively. In the natural sciences, this complementary relationship is crucial because conducting laboratory experiments for every conceivable environmental scenario is impractical. However, simulation models also require laboratory data to accurately represent real-world conditions and validate their predictions. This interdependence ensures that simulations are grounded in reality while extending the reach of experimental findings. The field of geochemistry exemplifies the essential interplay between laboratory experiments and simulations.

The rhizosphere is a good example of a problem currently under investigation by scientists that show the merits of thermodynamic modelling (Northover et al., 2022). The rhizosphere is the narrow region of aqueous soil surrounding plant roots, and it has chemical properties that can differ drastically from bulk soil or water. Its small size, ranging from just a few micrometres to several millimetres, makes direct measurement challenging. In such cases, simulations are invaluable for modelling conditions that are otherwise too complex or constrained to study experimentally.

Steefel et al. (2015) outline a few operations modern geochemical software are capable of such as reactive transport modelling, surface exchange calculations, inverse modelling, and speciation determination. Scientists exploring the rhizosphere might want to simulate the concentration of species in the rhizosphere while changing parameters such as temperature, salinity, pH, and oxidation states. This type of simulation is an example of speciation calculations, which will be the focus of this paper.

Table 1 summarizes several options for speciation calculations and their features. Among these, PHREEQC stands out as a notable example. Developed by the United States Geological Survey (USGS), PHREEQC is a free and open-source software that is under active development. It offers robust user support through an online forum, where users can seek help and get answers to their software queries. PHREEQC is compatible with several scripting languages, including MATLAB, Fortran, and Python, while also being accessible to a wider audience through its graphical user interface (GUI) (Parkhurst, 2013).

These features make PHREEQC an excellent choice for geochemist looking to preform speciation simulations.

One limitation of PHREEQC, and indeed geochemical software in general, is database management. Instead of providing a source of truth, PHREEQC allows organizations to independently develop databases. This flexibility allows laboratories to create databases that solve specific needs, but also permits conflicting aqueous modelling frameworks. To better understand the challenges inconsistent databases create for aqueous modelling, it is important to first understand the relationship between user-provided inputs and PHREEQC-provided data.

Table 1: Comparison of different geochemical simulation software. X represents the presence of a given feature.

SOFTWARE	FREE	OPEN SOURCED	ACTIVE DEVELOPMENT	USER SUPPORT	MODERN GUI	SCRIPTING CAPABILITIES
PHREEQC	X	X	X	X	X	X
GEOCHEMIST WORKBENCH			X	X	X	
VISUAL MINTEQ	X	X	X	X	X	
WATEQF4	X	X				X

Geochemical Software and their Databases

PHREEQC relies on user-provided textual input files to define simulation parameters through specific BLOCKS. In each BLOCK, users can set various aspects of the simulation; such as pH, temperature, species, and concentrations. The software then accesses predefined databases, reading the BLOCKS for elements and chemical equations to perform chemical speciation simulations. The two most germane BLOCKS to speciation calculations are SOLUTION_MASTER_SPECIES (SMS) and SOLUTION_SPECIES (SS). These BLOCKS are present in PHREEQC databases but can be overwritten by input files.

Example aqueous modelling database entries in PHREEQC can be seen in code block 1. The SMS BLOCK in PHREEQC lists elements as chemical species. The Element Ag could be represented by the species Ag+ and As(+3) could be represented as $H_2AsO_3^-$. The block also defines attributes like alkalinity and gram formula weight (GFW). The SMS BLOCK distinguishes between primary species, which have no ionic charge, and secondary species, which have a charge and must be associated with a corresponding primary species. This distinction bears consideration because primary and secondary species are defined differently in the SOLUTION_SPECIES (SS) BLOCK. The SS BLOCK contains chemical equations that use SMS species to define the chemical, physical, and thermodynamic properties required for calculations. Key parameters, such as the log K value, which represents the logarithm of the equilibrium constant (the ratio of products to reactants), are derived from these equations. Primary SMS species must have an equation where the species appears on both sides of the equals sign with a log K value of 0.0, while secondary SMS species must have an equation where the species is the first on the left side of the equals sign. Other equations, considered aqueous species equations, do not need to be defined in the SMS BLOCK. For more detailed attributes and examples of these BLOCKS, refer to the PHREEQC user manual (Parkhurst, 2013).

```
SOLUTION_MASTER_SPECIES                                                    (1)
# Primary SMS
Ag   Ag+        0.0 Ag      107.8682
# Secondary SMS
Ag(1)    Ag+        0   Ag

SOLUTION_SPECIES
# Primary SMS equation definition
Ag+ =  Ag+
    log_k 0
# Secondary SMS equation definintion
 1.0000 H+ + 1.0000 Ag+ + 0.2500 O2  =  Ag+2 +0.5000 H2O
        log_k            -12.1244
```

A collection of SOLUTION_MASTER_SPECIES (SMS), SOLUTION_SPECIES (SS), and other PHREEQC blocks are contained in database (.dat) files. Several databases come pre-installed with PHREEQC, and it is up to the user to choose the database that best suits their simulation goals. Each database is unique and developed independently, allowing for flexibility in addressing specific geochemical problems. For instance, the LLNL database, developed by Lawrence Livermore National Laboratory, reflects the lab's primary focus on U.S. defence and nuclear deterrence (LLNL, 2024). As a result, the LLNL database includes highly detailed and accurate equations related to nuclear chemistry.

The Coding Challenge

Challenges in this paradigm arise when considering what constitutes an ideal database for speciation. A simulation in PHREEQC is limited by the number of equations defined in the SS BLOCK. Increasing the number of equations can enhance the simulation's ability to approximate real-world conditions, thereby increasing accuracy. Therefore, it is advantageous to use a database with the maximum number of relevant equations. A naive solution might attempt combining SMS and SS blocks from different databases to achieve greater equation depth and species coverage. However, this approach is not feasible. PHREEQC requires internal consistency between element-species definitions and SMS-SS definitions within a single database, while allowing differences among separate databases.

For example, consider the Tipping Hurley and Minteq databases, both of which come pre-installed with PHREEQC and are considered "general" databases. These databases have a broad range of elements, species, and equations in their SMS and SS blocks, aiming to address basic geochemical environments. In the Tipping Hurley database, iron is defined in the SMS block as:

```
Element  Species        Alkalinity    gfw|formula     gfw                          (2)
Fe       Fe+2           0.0           55.847          55.847
Fe(+2)   Fe+2           0.0           55.847
Fe(+3)   Fe+3           -2.0          55.847
```

In the database from Minteq:

```
Element  Species    Alkalinity  gfw|formula     gfw                              (3)
Fe       Fe+3       -2.0    Fe  55.847
Fe(2)    Fe+2       0.0 Fe
Fe(3)    Fe+3       -2.0    Fe
```

The aqueous model's database defines the element Fe with different ions. If these SMS blocks were copy-pasted into the same database file, the database would fail to compile due to the ambiguity of the iron element-species definitions. This is the type of discrepancy the user manual mentions and something this project attempts to resolve.

The flexibility allowed in defining elements fundamentally inhibits combining databases, but even when databases agree on element-species definitions, other conflicts can arise in the SS block. Hundreds of equations exist across multiple databases but are be defined with conflicting parameters, further complicating any attempts to merge databases effectively.

Aims and Objectives

The aim of this project creates an optimized PHREEQC database tailored for speciation calculations. This solution overcomes element-species definition differences among databases and conflicting equation attributes.

To achieve this, the SOLUTION_MASTER_SPECIES (SMS) and SOLUTION_SPECIES (SS) blocks will be expanded by scanning seven PHREEQC-provided databases and consolidating their elements and equations into a unified master database. The project involves examining each database to determine the number of species and equations they contain. A comparative analysis will be performed to identify conflicts among the databases. Based on this analysis, a baseline database will be established. The remaining databases will be reviewed for any information not included in the baseline database, and missing data will be appended from auxiliary databases to enhance the baseline database.

Methods

This section outlines the process for creating a combined PHREEQC database optimized for speciation calculations and conducting a comparative analysis of existing databases. The methodology involves several key stages: data extraction, cleaning, analysis, consolidation, and generation. These stages are designed to transform and integrate diverse PHREEQC databases into a unified format, allowing for direct comparison and analysis of their elements and equations. The process is implemented in Python, utilizing several additional packages outside the Python Standard Library, as listed in Table 2. The latest packaging and workflows standards were used to ensure the feasibility of future upkeep (Jolowicz, 2024, (*PEP 8 – Style Guide for Python Code*, n.d.). The most significant external dependency is PHREEQCPython, a Python wrapper for the open-source C++ PHREEQC libraries(Charlton & Parkhurst, 2011). This library is not actively maintained and requires Python version 3.1X to function. Other PHREEQC wrappers, such as PyPHREEQC, were considered but not chosen due to similar issues with active development and compatibility with older Python versions. The modules for database compilation can be found in the package `build_database`, while database analysis is in the `results` package.

Table 2: Package requirements

Package	Minimum Version	Purpose
Numpy	2.0	Numerical Programming
Pandas	2.2	Data Frame Management
Click	8.0	User Argument Parser
PHREEQCPython	1.5.2	Thermodynamic Aqueous Simulation Package
Seaborn	0.13	Data Visualization
Pytest	8.0.0	Testing suite

Data Processing Pipeline

Figure 1 provides an outline of the process for converting PHREEQC database files into DataFrame style objects, refactoring the data for consistency, and producing a unified master database. The parser module preforms an initial pass of the text file and constructs Pandas DataFrames (Pandas Documentation, 2.2.2). While the `parser_dat` module does preform some basic data standardization, its primary goal is to move from text to DataFrames. The `clean_tables` modules examines the SMS and SS DataFrames in fine-tunes them, coalescing into a unified format optimized for inter-database comparison.

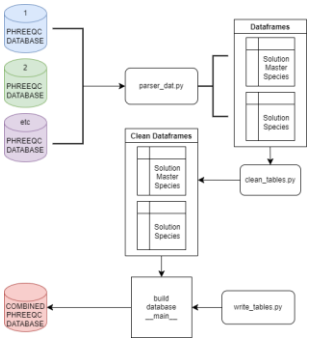


Figure 1: Outline of Python Package. Data is extracted from databases, cleaned, and written back into databases formatted for the PHREEQC software package.

A. Data Extraction and Conversion

The first step involves reading and parsing PHREEQC database files. The `parser_dat.py` module opens each database file and reads it line by line, using regular expressions (Regex) to detect key PHREEQC block keywords like SOLUTION_MASTER_SPECIES and SOLUTION_SPECIES. This allows the script to handle variations in block locations across different databases. Code block 4 outlines this process.

```
open file                                                                    (4)
for each line in file:
    if regex matches "SOLUTION_MASTER_SPECIES":
        start reading SMS block
    if regex matches "SOLUTION_SPECIES":
        start reading SS block
```

Once a block is identified, the script extracts relevant data, standardizing different formats using regular expressions to capture species names, chemical formulas, log K values, and other attributes. The data is then cleaned by removing extraneous characters and ensuring consistent formatting before being appended to lists for further processing.

Pseudocode:

```
for line in SMS block:                                                       (5)
    use regex to extract species name, formula, and attributes
    clean extracted data (remove spaces, format numbers, etc.)
    append cleaned data to SMS list

for line in SS block:
    use regex to extract chemical equation and log K value
    clean extracted data
    append cleaned data to SS list
```

After extraction and cleaning, the script converts the lists into Pandas DataFrames, which are then passed to the `clean_tables.py` module for further refinement.

B. Data Cleaning and Standardization

The `clean_tables.py` module further refines data by coalescing it into a unified format. This is achieved through a combination of Pandas DataFrame and Series methods (as depicted in Figure 2) and custom functions. Below is a brief description of several custom functions used in this process:

Key Functions

- replace_elements: Standardizes the notation of element charges across different databases by explicitly adding a '+' sign for positive charges, addressing variations such as Al(3) versus Al(+3).
- log_k_to_float: Converts log K values from strings to floating-point numbers, accommodating variations like extra annotations that make direct conversion impractical.
- replace_charges: Standardizes the representation of multiple negative charges by converting sequences of dashes (e.g., "----") into their numerical equivalents (e.g., "-4").
- clean_tuple: Cleans tuples by removing elements containing specified substrings (like 'vm' or 'dw') and converting valid numeric strings to floats, ensuring data is ready for numerical processing.

Each of these functions ensures that the data is clean, consistent, and ready for further analysis and integration into the final PHREEQC database.

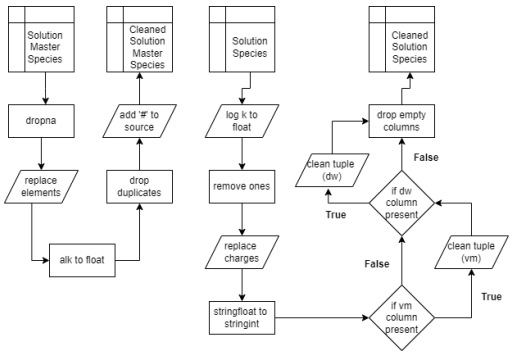


Figure 2: Data cleaning pipeline of SMS and SS DataFrames. Rectangles represent vectorized pandas functions, parallelograms are custom apply functions, and rhombuses are logic gates.

C. Database Compilation

The `build_database` package compiles PHREEQC databases into a master database optimized for speciation calculations and internal consistency. The `__main__.py` script assigns a ranking to each database, referred to as 'sources' in the code, through a predefined dictionary, determining the order in which databases are added to the final product.

If a database appears that is not in the ranking dictionary, it is given a rank of 5. The appending order among the 5th ranked sources is random. Justification of the ranking system is provided in the results and discussion section. The `compile_and_rank_mst` utility function is used to create a solution master species table (mst) by integrating the ranked databases, while the `compile_solution_species_table` function from the `clean_tables` module consolidates solution species from all databases into a unified table.

The database with rank one is assigned as a baseline table. The script then iterates through the list of auxiliary sources to identify any missing species. For each cleaned table, it checks for species not present in the current table using the `get_missing_species` function. If missing species are found from Minteq, the script applies additional processing via `process_missing_species` to address database-specific nuances.

The script also searches for missing equations from solution species tables using a Python lambda function that calls `find_and_collect_matches`. All matching indexes are collected, and unique entries are extracted. Certain chemical equations containing undesired species, identified using a regular expression, are excluded from the final database to ensure successful compilation.

Finally, the compiled solution master species and solution species tables are sorted by element and saved to an output file, specified by the user through command-line arguments. The utility function `save_master_database` writes the combined data to a specified file path, completing the database consolidation process.

Data Analysis

A comparative analysis of the PHREEQC-provided databases was performed using the `results` package, with findings documented in a Jupyter Notebook included in the package. The `clean_tables` module was utilized to import the databases and split them into Solution Master Species and Solution Species Pandas DataFrames, enabling thorough exploratory data analysis (EDA). While various functions and plots were implemented during EDA, the methodology for detecting conflicting data among source databases is particularly important and warrants detailed explanation.

Solution Master Species (SMS)

To identify conflicting entries in the SMS DataFrame, the data was filtered using the duplicated method from Pandas, focusing specifically on the 'element' column. This filtering process returns entries with the same value in the 'element' column but differing values in at least one other column. Entries with identical information across all columns were removed earlier in the process, meaning this method highlights discrepancies. An example of the results from this filtering process is shown in Table 3.

Table 3: The element column contains identical values for all entries, but each entry has at least one differing value in another column. For example, the second entry has a unique 'alk' value, while the third entry defines Acetate in its protonated form. Additionally, all entries have unique gram formula weights. The 'gfw_formula' and 'source' columns are ignored for this analysis.

element	species	alk	gfw_formula	element_gfw	source
Acetate	Acetate-	0.0	Acetate	59.01	sit.dat
Acetate	Acetate-	1.0	59.045	59.045	minteq.v4.dat
Acetate	HAcetate	0.0	Acetate	59.	llnl.dat

The `alk_gfw_duplicates` function examines the SMS DataFrame and alerts to duplicated entries based on a specified target column, such as alkalinity (alk) or gram formula weight (element_gfw). By focusing on these attributes, `alk_gfw_duplicates` can highlight variations in element definitions. For example, calling the `alk_gfw_duplicates` function and specifying the alk or element_gfw column would return the first and second entries in Table 3. The third entry would not be returned because it has a different species definition. Different alkalinity values make sense because protonated and deprotonated species are considered, but shouldn't the gram formula weight (gfw) be the same if the species only differ by a proton? While this is the case in Table 3, there are many instances where the species are drastically different, necessitating different molecular weights. Thus, only instances where element and species are identical count as a conflict for alk and gfw.

Solution Species

While a similar overall strategy was used in investigating the SS block, more careful consideration of the subtle difference between database equations was required. Although most equations have been standardized by `clean_tables`, minor variations in syntax remain and needed to be handled separately.

One instance of such subtleties is spacing in equation lines. To avoid parsing failures in PHREEQCPython and maintain readability, spacing and other text inconsistencies are not removed in `clean_tables` but are instead addressed in the results module. The results module allows for manipulation without affecting dependent modules. Once these alterations are complete, the SS DataFrame is grouped by the equation column, and a range function is applied to calculate the difference between the maximum and minimum log K values for each equation. Further analyses are conducted to identify equations with the most conflicts, the widest range of log K values, and sources with the most discrepancies.

Testing and Validation

A Pytest framework was developed to test code at the function, module, and expected outputs levels. For the `parser_dat.py` and `clean_tables` modules, code was written and then tests were developed to test code behaviour. For remaining modules, tests were first written with expected outputs and behaviours in mind and then code was developed to meet goals defined by tests. A combination of hypothetical and real data was used during testing depending on the situation. Testing coverage was calculated by the `coverage` external library and reads 94% test coverage.

To ensure calculations between the desktop version of PHREEQC and the wrapper PHREEQCPython returned comparable results, example problems from Bullen et al. (2024) were used and their results were compared. Speciation concentrations were verified to the limitations of machine precision. These and other example problems were used to validate calculations using the master database.

Results and Discussion

This project represents the first known comparison of PHREEQC databases, providing valuable insights for future geochemical research. Value counts of elements and equations are provided in Figure 4. We also developed new tools for converting databases into various formats, such as Pandas DataFrames, CSV, and JSON files. These formats lend themselves to automated procedures and scripting languages, allowing for in-depth comparison.

Utilizing these tools, we created a novel database that incorporates more elements and equations than any database currently provided by PHREEQC. This new database was verified for compatibility with both PHREEQCPython and the desktop version of PHREEQC, ensuring its utility across multiple platforms. The outputs from the novel database are within the same order of magnitude as those from existing databases, yet sufficiently distinct to justify further investigation.

These findings can guide PHREEQC developers in identifying common database conflicts and provide recommendations to authors regarding element-species definitions and equation parameters. By focusing on areas with the greatest discrepancies, developers can more effectively target outdated or incorrect information, thereby improving database accuracy. Furthermore, these results can assist users in selecting the most appropriate database for their specific geochemical modelling needs, based on the comparative analysis of different databases and their unique attributes.

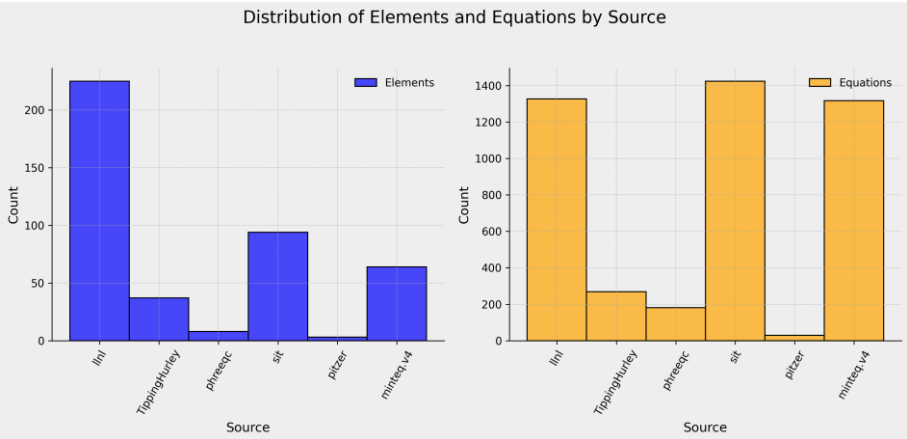


Figure 3: Histogram of elements (left) and equations (right) by source.

Analysis

Conflicts in Solution Master Species

When examining all seven databases investigated in this project, 107 elements defined in the SMS block were found to have conflicts. This indicates that an element defined in one database has a distinct species, alkalinity, or gram formula weight in another database. If an element has multiple conflicting definitions across several databases, it is still counted as a single conflict. To provide a more detailed analysis of these conflicts, we examined the differences in alkalinity, gram formula weight (GFW), and species definitions.

Attribute Conflicts: Alkalinity and GFW

In total, 18 instances were found where elements are defined with the same species but have different alkalinity values. The largest range of alkalinities was observed between the LLNL and SIT databases for hafnium (Hf), where LLNL defines Hf+4 with an alkalinity of 0.0, while SIT assigns an alkalinity of -4.0. Although this is an outlier, a pattern emerges with most alkalinity conflicts originating from the LLNL database (77.78%) and the SIT database (66.67%).

For the gram formula weight (GFW) attribute, 34 instances of conflict were identified among the databases. Upon examining the range for each element, 76% of the GFW differences were less than or equal to 0.01. However, there are significant discrepancies, such as for the "Alkalinity" species, where Minteq and Tipping Hurley report GFWs of 61.0173 and 50.05, respectively, for CO3-2. According to the National Center for Biotechnology Information (2024), the correct GFW for CO3-2 is 60.009, indicating a potential error in the Tipping Hurley database.

Species Conflicts

The most challenging aspect of unifying the SMS block involves the flexibility in defining elements with different species representations. A total of 35 elements were found to have conflicting species definitions across the databases. When attempting to create a unified database, it is necessary to choose a single schema for each element, as retaining multiple species definitions would cause the database to fail to compile. While attempts were made to develop a new naming convention to increase equation depth, this would require users to be familiar with each conflict and make choices at the elemental level. Ultimately, the LLNL schema was chosen due to its broad representation of elements.

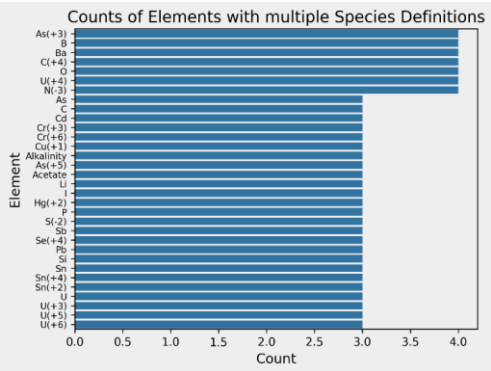


Figure 4: Histogram of elements with high disagreement among databases. Only species with 3 or more element-species conflicts are considered.

As an advisory to users, shown in Figure 5 are elements with species defined differently in three or more databases. If users have these elements in their simulation, care should be taken to ensure the best possible species definition is used to collect the most relevant equations.

Conflict in Solution Species

When examining the SS block, differences in log K values were focused on due to their importance in speciation calculations. There are 302 instances where databases report different log K values for the same equation. To facilitate understanding in how subtle these differences can be, raw equations from LLNL and SIT are shown in code block 6.

Commented [WJ1]: that needs ti be defined

```
# LLNL                                                                    (6)
3.0000 SO4-- + 1.0000 Zr+4  =  Zr(SO4)3--
        -llnl_gamma           4.0
        log_k           +7.3007
    -delta_H    0           # Not possible to calculate enthalpy of reaction    Zr(SO4)3-2
#   Enthalpy of formation:  -0 kcal/mol
# SIT
3.000SO4-2     + 1.000Zr+4     = Zr(SO4)3-2
     log_k    14.300     #05BRO/CUR
     # delta_h  0.000     #kJ/mol
     # Enthalpy of formation:                    #kJ/mol
     -analytic 1.43E+1 0E+0 0E+0 0E+0 0E+0
```

LLNL and SIT have different syntax blueprints, making these two equations seem different at first glance. `parser_dat.py` and `clean_tables.py` enable equations like these to be detected as the same equation.

This equation was chosen for its range in log K, 6.9993. For log values, this seems too large. However, the presence of the 'llnl_gamma' parameter in the LLNL equation acts as an operator on the equilibrium constant, changing the thermodynamics of the simulation. How different are the equations actually? The only way to tell is to run head-to-head simulations like those present in Figure 7. For the purposes of this paper, equations with 'llnl_gamma' parameters are included in the 302 duplicates found, but are excluded from figure 6. This figure reports equations with the largest range in log K values. These log K values should be further investigated for possible errors.

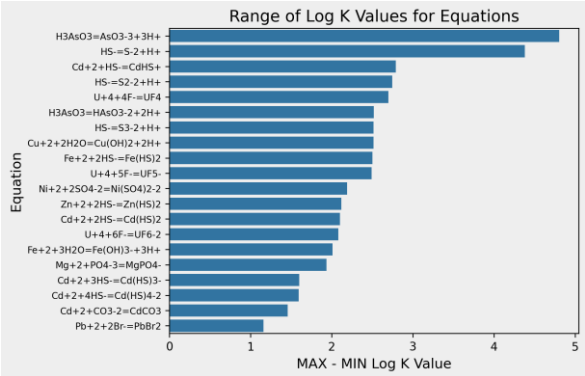


Figure 5: Range of Log K values. Sorted for largest found ranges. Equations with the 'llnl_gamma' parameter are ignored.'

Novel Database and Tool Development

This project successfully created a novel PHREEQC database containing 284 elements and 2,272 unique equations. The LLNL database contributes all its 1,327 equations, as expected for a baseline database. Each subsequent database contributes fewer elements and equations, which is logical since the likelihood of encountering a unique value decreases as the number of baseline values increases. Notably, all information relevant to low-salinity speciation calculations in the PITZER database is already available in the LLNL database, resulting in no additional data being added to the master database from PITZER.

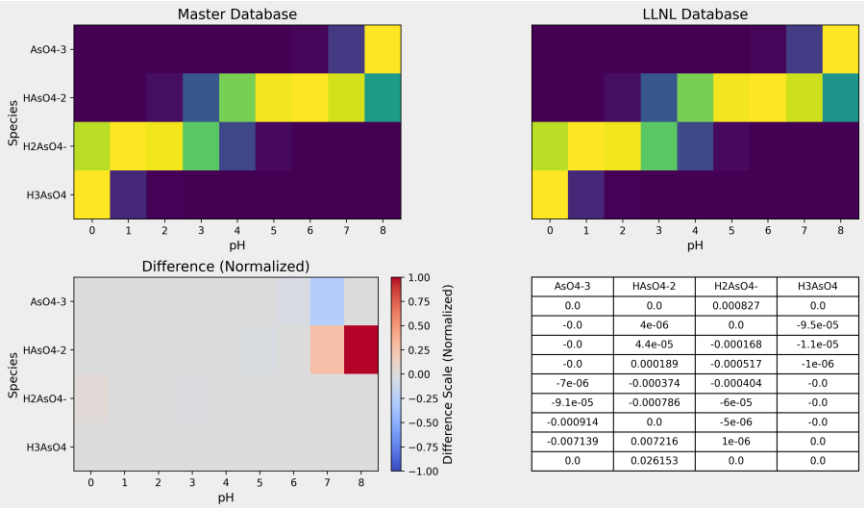
To choose the ranking of the databases, several factors were considered. The master database prioritizes general coverage, so total number of species and equations was important. The ratio of elements to equations was also considered to avoid prioritising databases with a deep but narrow focus, like SIT. LLNL stands out as a baseline database because if its excellent elemental and equation coverage. To rank the remaining databases, the geometric and harmonic means of elements, equations, and their ratio was calculated. Their agreement resulted in ranking shown in code block 7

```
rank = {                                                    (7)
    '#llnl.dat': 1,
    '#minteq.v4.dat': 2,
    '#phreeqc.dat': 3,
    '#Tipping_Hurley.dat': 4,
}
```

To ensure compatibility across platforms, simulations were run using both the desktop and Python implementations of PHREEQC, as outlined in the Testing and Validation subsection of the Methods section. Additionally, a speciation exercise from Bullen et al. (2024) was conducted to verify that the

developed master database computes results that are similar but not identical to those from existing databases. Figure 7 illustrates the results of these calculations. As the pH increases from 0 to 8, the concentration of acidic species decreases while the concentration of basic species increases, which is the expected behaviour.

The difference map in Figure 7 indicates that the databases yield their most divergent results at higher pH levels. This divergence could be attributed to conflicting alkalinity values in the SMS block. Notably, there are more conflicting alkalinity values for species with a negative ion than for neutral or positive ions combined. These findings highlight the importance of careful consideration of database conflicts in high-pH environments to improve the accuracy of geochemical modelling.



AsO4-3	HAsO4-2	H2AsO4-	H3AsO4
0.0	0.0	0.000827	0.0
-0.0	4e-06	0.0	-9.5e-05
-0.0	4.4e-05	-0.000168	-1.1e-05
-0.0	0.000189	-0.000517	-1e-06
-7e-06	-0.000374	-0.000404	-0.0
-9.1e-05	-0.000786	-6e-05	-0.0
-0.000914	0.0	-5e-06	-0.0
-0.007139	0.007216	1e-06	0.0
0.0	0.026153	0.0	0.0

Figure 6: Plot shows the normalized differences between the LLNL and novel Master Database with subtle differences emphasized. The heatmap on the bottom left highlights these differences with a diverging colour map. The bottom right shows absolute difference of values rounded by 6 digits.

Limitations

While this project has made significant strides in improving the consistency and comprehensiveness of databases for geochemical modelling, several limitations should be acknowledged:

- **Absence of PHASES Block**: The master database developed in this study does not currently include the PHASES block. This omission could limit the accuracy of certain simulations, particularly those involving phase equilibria, where mineral precipitation and dissolution are critical factors.
- **Regex Limitations in Keyword Parsing**: The regex patterns used in the database parser are designed to recognize all keywords currently employed in the seven databases analysed in this study. However, these patterns do not account for every possible form of keywords that the PHREEQC software allows. As a result, there may be unrecognized keywords in other databases, potentially leading to incomplete data extraction and integration if more databases are added to the package.
- **Limited Validation Simulations**: The validation of the novel database was conducted using a limited set of simulations. To ensure the broader applicability and robustness of the database, a more extensive range of validation simulations should be performed. This would help to identify

any remaining inconsistencies or errors and improve the overall reliability of the database for diverse geochemical modelling scenarios.

These limitations indicate areas for further improvement and suggest directions for future research.

Conclusion

This research addresses the challenge of achieving internal consistency within the thermodynamic data and aqueous species definitions used in PHREEQC, a widely utilized geochemical modelling software. By developing a Python package to consolidate and standardize data from multiple PHREEQC databases, this project enhances the accuracy and reliability of speciation calculations.

The novel database created through this project incorporates a broader range of elements and equations than any single existing PHREEQC database, providing a more comprehensive tool for geochemical simulations. The comparative analysis of different databases highlighted significant conflicts in species definitions and thermodynamic parameters, emphasizing the need for standardized data practices in geochemical modelling. Additionally, the validation process confirmed that the new database performs consistently across different PHREEQC implementations, reinforcing its utility and reliability.

Future work will focus on further refining the Python package to handle more complex data integration scenarios and expanding the database to include additional species and reactions. By continuing to improve the consistency and comprehensiveness of geochemical databases, researchers can enhance the predictive power of simulation tools like PHREEQC, ultimately contributing to a deeper understanding of complex environmental processes.

References

Bullen, J. C., Landa-Cansigno, O., & Weiss, D. J. (2024). Teaching Adsorption Chemistry by Constructing Surface Complexation Models (SCM) in PHREEQC. *Journal of Chemical Education*, *101*(5), 1914–1924. https://doi.org/10.1021/acs.jchemed.3c01195

Charlton, S. R., & Parkhurst, D. L. (2011). Modules based on the geochemical model PHREEQC for use in scripting and programming languages. *Computers & Geosciences*, *37*(10), 1653–1663. https://doi.org/10.1016/j.cageo.2011.02.005

Jolowicz, C. (2024). *Hypermodern Python tooling: Building Reliable Workflows for an Evolving Python Ecosystem*.

National Center for Biotechnology Information (2024). PubChem Compound Summary for CID 19660, Carbonate Ion. Retrieved August 29, 2024 from https://pubchem.ncbi.nlm.nih.gov/compound/Carbonate-Ion.

Northover, G. H. R., Mao, Y., Blasco, S., Vilar, R., Garcia-España, E., Rocco, C., Hanif, M., & Weiss, D. J. (2022). Synergistic use of siderophores and weak organic ligands during zinc transport in the rhizosphere controlled by pH and ion strength gradients. *Scientific Reports*, *12*(1). https://doi.org/10.1038/s41598-022-10493-5

Lawrence Livermore National Laboratory (LLNL). (n.d.). https://www.llnl.gov/

pandas documentation — pandas 2.2.2 documentation. (n.d.). https://pandas.pydata.org/docs/index.html

Parkhurst, D.L., and Appelo, C.A.J., 2013, Description of input and examples for PHREEQC version 3—A computer program for speciation, batch-reaction, one-dimensional transport, and inverse geochemical calculations: U.S. Geological Survey Techniques and Methods, book 6, chap. A43, 497 p., available only at http://pubs.usgs.gov/tm/06/a43.

PEP 8 – Style guide for Python code | peps.python.org. (n.d.). Python Enhancement Proposals (PEPs). https://peps.python.org/pep-0008/

Steefel, C. I., Appelo, C. a. J., Arora, B., Jacques, D., Kalbacher, T., Kolditz, O., Lagneau, V., Lichtner, P. C., Mayer, K. U., Meeussen, J. C. L., Molins, S., Moulton, D., Shao, H., Šimůnek, J., Spycher, N., Yabusaki, S. B., & Yeh, G. T. (2014). Reactive transport codes for subsurface environmental simulation. *Computational Geosciences*, *19*(3), 445–478. https://doi.org/10.1007/s10596-014-9443-x